



```
#include<bits/stdc++.h>
#include<iostream>

using namespace std;
//function that calculate modular exponentiation x^n mod m.
int modpower(int x, int n, int m)
{
    if (n == 0) //base case
        return 1%m;
    long long u = modpower(x,n/2,m);
    u = (u*u)%m;
    if (n%2 == 1) //when 'n' is odd
        u = (u*x)%m;
    return u;
}
//driver function
int main()
{
    cout<<modpower(5,2,7)<<endl;
    return 0;
}
```

Explain this code

Got it  Let's go through this code step by step.

◆ Purpose of the Code

This program calculates **modular exponentiation**, i.e. it computes:

$$(x^n) \bmod m$$

efficiently using **divide-and-conquer (recursive fast exponentiation)**.

◆ Code Breakdown

cpp

```
#include<bits/stdc++.h>
#include<iostream>
using namespace std;
```

- Includes standard libraries. (**bits/stdc++.h** already includes **<iostream>**, so including both is redundant.)

Function: modpower(int x, int n, int m)

cpp

```
int modpower(int x, int n, int m)
{
    if (n == 0) //base case
        return 1%m;
```

- **Base case:** Any number raised to power 0 is 1.
- But since we are taking modulo m, it returns 1 % m.

cpp

```
long long u = modpower(x,n/2,m);
u = (u*u)%m;
```

- The function calls itself recursively with n/2.
- This is the **divide step**: instead of directly computing x^n , it computes $x^{(n/2)}$ first.
- Then squares it:

$$u = (x^{n/2}) \times (x^{n/2}) \bmod m$$

This reduces the number of multiplications drastically (logarithmic time instead of linear).

cpp

```
if (n%2 == 1) //when 'n' is odd
```

```
u = (u*x)%m;
```

- If n is odd, then:

$$x^n = x^{n/2} \cdot x^{n/2} \cdot x$$

- So we multiply by x one extra time.

```
cpp
    return u;
}
```

- Returns the final result modulo m .

Main Function

```
cpp
int main()
{
    cout<<modpower(5,2,7)<<endl;
    return 0;
}
```

- Calls the function with:
 - $x = 5$
 - $n = 2$
 - $m = 7$

♦ Dry Run Example

We are calculating:

$$5^2 \mod 7$$

1. `modpower(5,2,7)`

- $n = 2 \rightarrow$ not zero
- recursive call $\rightarrow$ `modpower(5,1,7)`

2. `modpower(5, 1, 7)`

- $n = 1 \rightarrow$ not zero
- recursive call $\rightarrow \text{modpower}(5, 0, 7)$

3. `modpower(5, 0, 7)`

- base case $\rightarrow$ returns $1 \% 7 = 1$.

4. Back to $n=1$:

- $u = 1$
- $u = (1*1)\%7 = 1$
- since 1 is odd $\rightarrow u = (1*5)\%7 = 5$
- return 5.

5. Back to $n=2$:

- $u = 5$
- $u = (5*5)\%7 = 25 \% 7 = 4$
- $n=2$ is even $\rightarrow$ no extra multiply
- return 4.

✓ Output = 4

◆ Key Notes

- **Time Complexity:** $O(\log n)$ (much faster than naive $O(n)$ multiplication).
- **Space Complexity:** $O(\log n)$ due to recursion stack.
- This algorithm is known as **Exponentiation by Squaring (Binary Exponentiation)**.

👉 Do you want me to also write the **iterative version (loop-based)** of this algorithm, which avoids recursion?



explain Chinese Remainder Theorem

Alright 👍 Let's go step by step and clearly explain the **Chinese Remainder Theorem (CRT)**.

♦ What is the Chinese Remainder Theorem (CRT)?

The **Chinese Remainder Theorem** is a result in number theory that helps solve a system of **simultaneous congruences** with **different moduli**.

It says:

If we have a system of congruences like:

$$x \equiv a_1 \pmod{m_1}$$

$$x \equiv a_2 \pmod{m_2}$$

$$x \equiv a_3 \pmod{m_3}$$

...

where the moduli $m_1, m_2, m_3, \dots$ are **pairwise coprime** (gcd of any two = 1),

then there exists a **unique solution modulo M** , where:

$$M = m_1 \times m_2 \times m_3 \times \dots$$

♦ Why is it useful?

Instead of solving a large modulus problem directly, CRT lets us **break it into smaller modular equations**, solve them separately, then combine the solutions.

This is very useful in **cryptography (RSA)**, computer arithmetic, and competitive programming.

♦ Step-by-step Method

Suppose we want to solve:

$$x \equiv a_1 \pmod{m_1}, \quad x \equiv a_2 \pmod{m_2}, \quad \dots$$

1. Compute the **product of all moduli**:

$$M = m_1 \times m_2 \times \cdots \times m_k$$

2. For each congruence:

- Compute $M_i = \frac{M}{m_i}$
(i.e., divide the total product by that modulus).

3. Find the **modular inverse** of M_i modulo m_i :

$$y_i = M_i^{-1} \pmod{m_i}$$

(This means $M_i \cdot y_i \equiv 1 \pmod{m_i}$).

4. Build the solution as:

$$x = \sum_{i=1}^k a_i \cdot M_i \cdot y_i \pmod{M}$$

◆ Example 1

Solve the system:

$$x \equiv 2 \pmod{3}$$

$$x \equiv 3 \pmod{4}$$

$$x \equiv 2 \pmod{5}$$

Step 1: Compute total product

$$M = 3 \times 4 \times 5 = 60$$

Step 2: Compute M_i

- $M_1 = 60/3 = 20$
- $M_2 = 60/4 = 15$
- $M_3 = 60/5 = 12$

Step 3: Find modular inverses

- Find y_1 such that $20y_1 \equiv 1 \pmod{3}$.
 $\rightarrow 20 \equiv 2 \pmod{3}$, so solve $2y_1 \equiv 1 \pmod{3}$.
 $\rightarrow y_1 = 2$.
- Find y_2 such that $15y_2 \equiv 1 \pmod{4}$.
 $\rightarrow 15 \equiv 3 \pmod{4}$.
 $\rightarrow$ Solve $3y_2 \equiv 1 \pmod{4}$.
 $\rightarrow y_2 = 3$.
- Find y_3 such that $12y_3 \equiv 1 \pmod{5}$.
 $\rightarrow 12 \equiv 2 \pmod{5}$.
 $\rightarrow$ Solve $2y_3 \equiv 1 \pmod{5}$.
 $\rightarrow y_3 = 3$.

Step 4: Build solution

$$x = a_1 M_1 y_1 + a_2 M_2 y_2 + a_3 M_3 y_3$$

$$x = (2)(20)(2) + (3)(15)(3) + (2)(12)(3)$$

$$x = 80 + 135 + 72 = 287$$

$$x \equiv 287 \pmod{60} \Rightarrow x \equiv 47 \pmod{60}$$

✓ Final Answer:

$$x \equiv 47 \pmod{60}$$

◆ Summary

- CRT helps solve multiple modular equations.
- Works only when moduli are **pairwise coprime**.
- Solution is **unique modulo the product of all moduli**.
- Formula:

$$x = \sum (a_i \cdot M_i \cdot y_i) \pmod{M}$$

👉 Do you want me to also show you a **C++ implementation** of CRT (general form), so you can solve these types of problems in code?



explain Fermat's Little Theorem

Perfect question 👍 Let's go through **Fermat's Little Theorem** step by step.

◆ Fermat's Little Theorem (FLT)

It is a fundamental result in **number theory** related to **prime numbers** and **modular arithmetic**.

It says:

👉 If p is a **prime number** and a is an integer such that a is **not divisible by p** (i.e., $\gcd(a, p) = 1$), then:

$$a^{p-1} \equiv 1 \pmod{p}$$

◆ Alternate Form

Multiply both sides by a :

$$a^p \equiv a \pmod{p}$$

This works for **all integers a** (even when a is divisible by p).

◆ Why is it Important?

1. Modular exponentiation simplification

Instead of calculating huge powers, we can reduce the exponent modulo $p - 1$.
Example: To compute $a^{1000} \bmod 7$, reduce exponent $1000 \bmod 6$ because $p - 1 = 6$.

2. Cryptography (RSA) relies on this theorem.

3. Primality testing: Fermat primality test.

◆ Proof Idea (Intuitive)

Let's take numbers:

$$1, 2, 3, \dots, (p-1)$$

Multiply each by $a \pmod p$:

$$a \cdot 1, a \cdot 2, a \cdot 3, \dots, a \cdot (p-1)$$

Because p is prime and $\gcd(a, p) = 1$, these products are just a **rearrangement** of $1, 2, 3, \dots, (p-1)$ modulo p .

So:

$$a \cdot 1 \cdot a \cdot 2 \cdot \dots \cdot a \cdot (p-1) \equiv 1 \cdot 2 \cdot \dots \cdot (p-1) \pmod p$$

That is:

$$a^{p-1} \cdot (p-1)! \equiv (p-1)! \pmod p$$

Cancel $(p-1)!$ (valid since it's not divisible by p):

$$a^{p-1} \equiv 1 \pmod p$$

✓ Proven.

◆ Example 1

Take $a = 2, p = 7$.

$$2^{7-1} = 2^6 = 64$$

$$64 \bmod 7 = 1$$

✓ Works.

◆ Example 2

Take $a = 3, p = 5$.

$$3^{5-1} = 3^4 = 81$$

$$81 \bmod 5 = 1$$

✓ Works.

◆ How It's Used in Computation

Suppose you want $7^{222} \bmod 13$.

Since $p = 13$ (prime):

$$a^{p-1} \equiv 1 \pmod{13}$$

$$7^{12} \equiv 1 \pmod{13}$$

Now reduce exponent $222 \bmod 12 = 6$.

$$7^{222} \equiv 7^6 \pmod{13}$$

$$7^6 = 117,649$$

$$117,649 \bmod 13 = 9$$

✓ Answer: $7^{222} \equiv 9 \pmod{13}$

Much easier than computing the full number.

◆ Summary

- If p is prime:

$$a^{p-1} \equiv 1 \pmod{p}, \quad \gcd(a, p) = 1$$

$$a^p \equiv a \pmod{p}, \quad \text{for all } a$$

- Helps with **fast modular exponentiation** and **cryptography**.
- Proof comes from properties of modular arithmetic and rearranging multiples.

👉 Do you want me to also show how **Fermat's Little Theorem** connects to your earlier code for **modular exponentiation** (to make large powers easier)?



finding nCr , modulo m ,

$$1000C_{500} \% m$$

$$\rightarrow (1000!) \% m$$

$$(1000-500)! \cdot 500! \% m$$

$$O(n)$$

$$1000C_{500} = \left[999C_{499} + 999C_{500} \right]$$

$$\rightarrow (1000!) \% m \cdot \underbrace{((1000-500)!)^{-1} (500!)^{-1} \% m}_{\text{inverse modulo with } m.}$$

Modular Arithmetic

Addition: $(a+b) \% m = (a \% m + b \% m) \% m$

Substitution: $(a-b) \% m = (a \% m - b \% m + m) \% m$

Multiplication: $(ab) \% m = (a \% m \cdot b \% m) \% m$ modular inverse of b

Division: $(a/b) \% m = (a \cdot b^{-1}) \% m = (a \% m \cdot b^{-1} \% m) \% m$
 $b \neq 0$ $b^{-1} \neq 1/b$

$a \% m = r \geq 0, \leq a$
 $r = a - \lfloor \frac{a}{m} \rfloor \cdot m$
 $-a \% m = -r$
 $= +m$

BigMod Calculation $x^y \% m$

Naive Approach:

$p = 1$
 for $(i = 1 \text{ to } y)$
 $p = (p * (x \% m)) \% m$

$O(y)$ can be done in $O(\log_2 y)$

$O(\log_2 y)$ approach:

$\Rightarrow x^y = x^{y/2} \cdot x^{y/2} \cdot x^{(y \% 2)}$

also called Divide and conquer approach.

$x^5 = x^2 \cdot x^2 \cdot x$

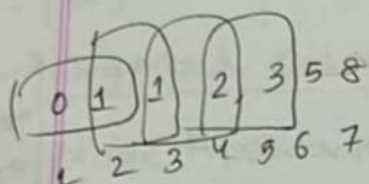
$-15 \% 4 = -3$
 $-15 = -4 * 4 + 1$
 $9 \mid \frac{-15}{-4} \mid 1$

$(x \cdot b) \% m$
 $x = b^{-1}$
 $b = 3, m = 5$
 $b^{-1} = 2$

$$f_{n+1} = f_n + f_{n-1}$$

$$f_n = f_{n-1} + f_{n-2}$$

$$\begin{bmatrix} f_n \\ f_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}^{(n-2)} \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$



$$\begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} f_n \\ f_{n-1} \end{bmatrix}$$

$$\begin{aligned} f_n + f_{n-1} &= f_{n+1} \\ f_n + 0 &= f_n \end{aligned}$$

$$\begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} f_{n+1} \\ f_n \end{bmatrix} =$$

$$\begin{aligned} f_{n+1} + f_n &= f_{n+2} \\ f_{n+1} &= f_{n+1} \end{aligned}$$

$$\text{ModMul } 2 \times 2 (M, N) \sum$$

$$\text{res}[0,0] = (M[0,0] * N[0,0] + M[0,1] * N[1,0]) \% M$$

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} p & q \\ r & s \end{bmatrix} = \begin{bmatrix} ap+br & aq+bs \\ cp+dr & cq+ds \end{bmatrix}$$

$$n^{\text{th}} = M^{n-2} \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

BigMod (x, y, m):

if $y == 0$ return 1;

$p = \text{BigMod}(x, \lfloor y/2 \rfloor, m)$

$P = (p * p) \% m$

if $(y \% 2)$

$P = (p * x) \% m$

return P;

Fibonacci Sequence:

0, 1, 1, 2, 3, 5, 8, 13, ...

Goal to find n th fibonacci modulo m :

Naive Approach:

$a = 0,$

$b = 1,$

for $i = 2$ to n ;

$b = (a + b) \% m$

$a = (b - a) \% m$

$O(n)$

$a = 2$

$b = 3$

$b = 2 + 3 = 5$

$a = 5 - 2 = 3$

Possible in $O(\log_2 n)$

can you tell me which topics are done in these pictures

Yes 🍌 I can identify the topics covered in your handwritten notes from the images.
Here's a structured list:

Topics Covered in Your Notes

1. Binomial Coefficient Modulo m

- $nCr \bmod m$ calculation
- Example: $\binom{1000}{500} \bmod m$
- Formula with factorials:

$$nCr = \frac{n!}{r!(n-r)!}$$

- Using **modular inverse** for division in modular arithmetic.
 - Concept of inverse modulo with Fermat's Little Theorem.
-

2. Modular Arithmetic Rules

- Addition: $(a + b) \% m$
 - Subtraction: $(a - b) \% m$
 - Multiplication: $(a \times b) \% m$
 - Division: requires **modular inverse** (not direct division).
 - Explanation of modular inverse.
-

3. Modular Exponentiation (BigMod)

- Problem: compute $x^y \bmod m$.
 - **Naive approach:** $O(y)$.
 - **Efficient approach (Exponentiation by Squaring):** $O(\log y)$.
 - Recursive divide-and-conquer method.
-

4. Matrix Method for Fibonacci Numbers

- Fibonacci recurrence:

$$F_{n+1} = F_n + F_{n-1}$$

- Matrix representation:

$$\begin{bmatrix} F_{n+1} \\ F_n \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} F_n \\ F_{n-1} \end{bmatrix}$$

- Use of **matrix exponentiation** to calculate Fibonacci efficiently.
- Matrix multiplication modulo m .
- Complexity: $O(\log n)$.

5. Fibonacci Sequence Modulo m

- Sequence: 0, 1, 1, 2, 3, 5, 8, 13...
- Goal: find n -th Fibonacci number $\text{mod } m$.
- Naive iterative approach: $O(n)$.
- Fast method using matrix exponentiation: $O(\log n)$.

Summary of Covered Topics

1. Binomial coefficient modulo m using modular inverse
2. Modular arithmetic operations (add, sub, mul, div)
3. Modular exponentiation (BigMod) – naive vs fast method
4. Matrix exponentiation for Fibonacci numbers
5. Fibonacci sequence modulo m – naive vs fast method

Do you want me to **make a clean summary sheet (like an exam cheat sheet with formulas + algorithms)** for these topics?



Binomial Coefficient Modulo m and give me pseudo code

Great question 👍 Let's carefully explain **Binomial Coefficient Modulo m** and then I'll give you a clean **pseudo code**.

◆ Binomial Coefficient Modulo m

We want to compute:

$$nC_r \bmod m = \frac{n!}{r!(n-r)!} \bmod m$$

⚠ Problem

- Direct division does **not work** in modular arithmetic.
- We need **modular inverse** of $r!(n-r)!$ under modulo m .

◆ Fermat's Little Theorem (when m is prime)

If m is prime, the modular inverse of a number a is:

$$a^{-1} \equiv a^{m-2} \pmod{m}$$

So:

$$nC_r \bmod m = (n! \times (r!)^{-1} \times ((n-r)!)^{-1}) \bmod m$$

◆ Pseudo Code (Prime Modulus)

python

```
function nCr_mod(n, r, m):
    if r > n:
        return 0

    # Step 1: Precompute factorials modulo m
    fact = array of size (n+1)
    fact[0] = 1
    for i = 1 to n:
        fact[i] = (fact[i-1] * i) % m

    # Step 2: Compute modular inverses using Fermat's Little Theorem
```

```

inv_r = pow(fact[r], m-2, m)      # (r!)^-1 mod m
inv_nr = pow(fact[n-r], m-2, m)  # ((n-r)!)^-1 mod m

# Step 3: Apply formula
result = (fact[n] * inv_r) % m
result = (result * inv_nr) % m

return result

```

◆ Explanation

1. **Precompute factorials** modulo m .
2. Use **modular exponentiation** (fast power) to compute modular inverses.
 - `pow(a, b, m)` means $a^b \bmod m$.
3. Plug into formula:

$$nC_r = \frac{n!}{r!(n-r)!} \bmod m$$

◆ Example

Find $1000C_{500} \bmod 13$.

- Compute factorials mod 13.
- Compute inverses of $500!$ and $500! \bmod 13$ using $a^{11} \bmod 13$.
- Multiply according to formula.

✓ This works efficiently in **$O(n + \log m)$** time.

👉 Do you also want me to give you a **C++ implementation** of this pseudo code (optimized for large n)?



explain Modular Arithmetic Rules

Perfect 👍 Let's carefully go through **Modular Arithmetic Rules** (these are the foundations for everything you saw in your notes).

♦ What is Modular Arithmetic?

When we write

$$a \equiv b \pmod{m}$$

it means a and b leave the **same remainder** when divided by m .

Example:

$$17 \equiv 5 \pmod{12} \quad \text{because } 17 \% 12 = 5$$

♦ Basic Modular Rules

1. Addition Rule

$$(a + b) \bmod m = ((a \bmod m) + (b \bmod m)) \bmod m$$

✓ Example:

$$(7 + 9) \bmod 5 = 16 \bmod 5 = 1$$

$$\text{Also: } (7 \bmod 5 + 9 \bmod 5) \bmod 5 = (2 + 4) \bmod 5 = 6 \bmod 5 = 1$$

2. Subtraction Rule

$$(a - b) \bmod m = ((a \bmod m) - (b \bmod m)) \bmod m$$

⚠ If the result is negative, add m to make it positive.

✓ Example:

$$(7 - 9) \bmod 5 = -2 \bmod 5 = 3$$

$$\text{Also: } (7 \bmod 5 - 9 \bmod 5) \bmod 5 = (2 - 4) \bmod 5 = -2 \bmod 5 = 3$$

3. Multiplication Rule

$$(a \times b) \bmod m = ((a \bmod m) \times (b \bmod m)) \bmod m$$

✓ Example:

$$(7 \times 9) \bmod 5 = 63 \bmod 5 = 3$$

$$\text{Also: } (7 \bmod 5 \times 9 \bmod 5) \bmod 5 = (2 \times 4) \bmod 5 = 8 \bmod 5 = 3$$

4. Division Rule (Tricky)

Division is **not directly possible** in modular arithmetic.

Instead, we multiply by the **modular inverse**.

$$\frac{a}{b} \bmod m = (a \times b^{-1}) \bmod m$$

where b^{-1} is the **modular inverse** of b modulo m :

$$b \times b^{-1} \equiv 1 \pmod{m}$$

- Modular inverse exists **only if** $\gcd(b, m) = 1$.
- If m is prime, use **Fermat's Little Theorem**:

$$b^{-1} \equiv b^{m-2} \pmod{m}$$

✓ Example:

Compute $\frac{7}{3} \bmod 11$.

- Find inverse of 3 mod 11 $\rightarrow 3^9 \bmod 11 = 4$.
- So: $7 \times 4 \bmod 11 = 28 \bmod 11 = 6$.

Answer = 6.

◆ Summary Table

Operation	Rule
Addition	$(a + b) \% m = ((a \% m) + (b \% m)) \% m$
Subtraction	$(a - b) \% m = ((a \% m) - (b \% m) + m) \% m$
Multiplication	$(a \times b) \% m = ((a \% m) \times (b \% m)) \% m$
Division	$(a/b) \% m = (a \times b^{-1}) \% m, \text{ needs inverse}$

👉 Would you like me to also make a **one-page formula sheet (PDF style)** with these modular rules, Fermat's theorem, and BigMod for your exam prep?



Modular Exponentiation (BigMod) and give me cpp code

Alright 👍 let's cover **Modular Exponentiation (BigMod)** clearly and then I'll give you the **C++ code**.

◆ Modular Exponentiation (BigMod)

We want to compute:

$$x^y \bmod m$$

Why not compute directly?

- If y is large (e.g., 10^9), x^y is astronomically huge.
- We need an efficient method to avoid overflow and reduce time.

◆ Naive Approach

Multiply x by itself y times and take modulo:

```
cpp
p = 1;
for (i=1; i<=y; i++) {
    p = (p * x) % m;
}
```

⌚ Complexity: $O(y) \rightarrow$ too slow for large y .

◆ Efficient Approach (Exponentiation by Squaring)

We use the property:

$$x^y = \begin{cases} (x^{y/2})^2 & \text{if } y \text{ is even} \\ (x^{y/2})^2 \times x & \text{if } y \text{ is odd} \end{cases}$$

This reduces complexity to:

$$O(\log y)$$

◆ C++ Code (Recursive)

```
cpp

#include <bits/stdc++.h>
using namespace std;

// Function to compute (x^y) % m
long long BigMod(long long x, long long y, long long m) {
    if (y == 0)
        return 1 % m;    // base case

    long long p = BigMod(x, y / 2, m);
    p = (p * p) % m;    // square step

    if (y % 2 == 1)    // if y is odd
        p = (p * x) % m;

    return p;
}

int main() {
    long long x = 5, y = 117, m = 19;
    cout << "Result: " << BigMod(x, y, m) << endl;
    return 0;
}
```

◆ C++ Code (Iterative → Faster, avoids recursion stack)

```
cpp

#include <bits/stdc++.h>
using namespace std;

long long BigMod(long long x, long long y, long long m) {
    long long result = 1;
    x = x % m;    // reduce base

    while (y > 0) {
        if (y % 2 == 1)    // if y is odd
```

```

        result = (result * x) % m;

        y = y / 2;          // halve exponent
        x = (x * x) % m;    // square base
    }

    return result;
}

int main() {
    long long x = 5, y = 117, m = 19;
    cout << "Result: " << BigMod(x, y, m) << endl;
    return 0;
}

```

◆ Example Run

Compute:

$$5^{117} \bmod 19$$

- Using BigMod → Output = 1

✓ So, **BigMod** is the efficient way to compute large powers modulo m in $O(\log y)$ time.

Do you want me to also explain how **BigMod + Fermat's Little Theorem** is used to compute **modular inverse** (for binomial coefficients like $nCr \bmod m$)?



explain Matrix Method for Fibonacci Numbers and go through an example

Great question 👍 Let's carefully go through the **Matrix Method for Fibonacci Numbers** and then solve an example step by step.

◆ Fibonacci Recurrence

We know the Fibonacci sequence:

$$F_0 = 0, \quad F_1 = 1$$

$$F_n = F_{n-1} + F_{n-2} \quad \text{for } n \geq 2$$

So:

0, 1, 1, 2, 3, 5, 8, 13, 21, ...

◆ Matrix Representation

We can represent the Fibonacci recurrence as a **matrix multiplication**:

$$\begin{bmatrix} F_{n+1} \\ F_n \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} \cdot \begin{bmatrix} F_n \\ F_{n-1} \end{bmatrix}$$

◆ General Formula

If we apply this repeatedly:

$$\begin{bmatrix} F_{n+1} \\ F_n \end{bmatrix} = \left(\begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} \right)^n \cdot \begin{bmatrix} F_1 \\ F_0 \end{bmatrix}$$

Since $\begin{bmatrix} F_1 \\ F_0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$,

we get:

$$\begin{bmatrix} F_{n+1} \\ F_n \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^n \cdot \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

So, F_n is just the **bottom-left element** of the power matrix.

◆ Example: Find F_5

We want the 5th Fibonacci number.

Step 1: Define matrix

$$M = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$$

Step 2: Compute M^{n-1}

We need M^4 (because F_5 is stored in that power).

Let's compute step by step:

- $M^1 = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$
- $M^2 = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} \cdot \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix}$
- $M^4 = (M^2)^2 = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix} \cdot \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 5 & 3 \\ 3 & 2 \end{bmatrix}$

Step 3: Multiply with base vector

$$M^4 \cdot \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 5 \\ 3 \end{bmatrix}$$

So:

- $F_5 = 5$
- $F_4 = 3$

✓ Matches the Fibonacci sequence: 0, 1, 1, 2, 3, 5.

◆ Complexity

- Naive method $\rightarrow O(n)$
- Matrix exponentiation with fast power $\rightarrow O(\log n)$

This is very useful for **huge** n (like 10^{18}).

◆ Summary

- Fibonacci can be written as a matrix power problem.
- Formula:

$$\begin{bmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^n$$

- Use **fast exponentiation** to compute in $O(\log n)$.

👉 Do you want me to also write the **C++ code for Fibonacci using matrix exponentiation modulo m** (like finding $F_n \bmod m$)?



$$3 \times 2 \equiv 1 \pmod{5}$$

what does this expression mean?

Nice question 👍

The expression

$$3 \times 2 \equiv 1 \pmod{5}$$

means:

- Multiply $3 \times 2 = 6$.
- When 6 is divided by 5, the remainder is 1.

$$6 \div 5 = 1 \text{ remainder } 1$$

So:

$$6 \equiv 1 \pmod{5}$$

♦ Modular Inverse Meaning

This also shows that **2 is the modular inverse of 3 modulo 5**, because:

$$3 \times 2 \equiv 1 \pmod{5}$$

By definition:

- If $a \times b \equiv 1 \pmod{m}$,
then b is the **modular inverse** of a modulo m .

So here:

- Inverse of $3 \bmod 5 = 2$
- Inverse of $2 \bmod 5 = 3$

✓ In simple words:

This expression means that **3 and 2 "undo" each other in modulo 5 arithmetic.**

👉 Do you want me to also explain **how to find modular inverses** in general (using Extended Euclidean Algorithm and Fermat's theorem)?



give me cpp code for Matrix Method for Fibonacci Numbers using recursion

Perfect 👍 Let's implement the **Matrix Method for Fibonacci Numbers** using **recursion (fast exponentiation by squaring on matrices)** in C++.

◆ Idea Recap

We use the transformation matrix:

$$M = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$$

Then:

$$F_n = (M^{n-1})[0][0], \quad \text{for } n \geq 2$$

We compute M^{n-1} using **recursive fast exponentiation**.

◆ C++ Code (Recursive Matrix Method)

cpp

```
#include <bits/stdc++.h>
using namespace std;
```

```
typedef long long ll;

// Function to multiply two 2x2 matrices modulo m
void multiply(ll A[2][2], ll B[2][2], ll m) {
    ll x = (A[0][0]*B[0][0] + A[0][1]*B[1][0]) % m;
    ll y = (A[0][0]*B[0][1] + A[0][1]*B[1][1]) % m;
    ll z = (A[1][0]*B[0][0] + A[1][1]*B[1][0]) % m;
    ll w = (A[1][0]*B[0][1] + A[1][1]*B[1][1]) % m;

    A[0][0] = x; A[0][1] = y;
    A[1][0] = z; A[1][1] = w;
}

// Recursive function to raise matrix M to power n modulo m
void power(ll M[2][2], ll n, ll m) {
    if (n == 0 || n == 1) return;

    ll base[2][2] = {{1,1},{1,0}};

    power(M, n/2, m);
    multiply(M, M, m);    // square step

    if (n % 2 != 0)        // if n is odd, multiply once more
        multiply(M, base, m);
}

// Function to compute nth Fibonacci number modulo m
ll fibonacci(ll n, ll m) {
    if (n == 0) return 0;
    if (n == 1) return 1 % m;

    ll M[2][2] = {{1,1},{1,0}};
    power(M, n-1, m);

    return M[0][0] % m;    // F(n) stored here
}

int main() {
    ll n = 10, mod = 1000;
    cout << "F(" << n << ") mod " << mod << " = " << fibonacci(n, mod) << endl;
    return 0;
}
```

◆ Example Run

Input: $n = 10$, $m = 1000$

Fibonacci sequence = 0,1,1,2,3,5,8,13,21,34,55...

- $F_{10} = 55$
- Output $\rightarrow 55$

✓ This code computes **Fibonacci numbers in $O(\log n)$** time using recursion.

👉 Do you want me to also show you an **iterative (non-recursive) version** of the same algorithm for comparison?



explain Fibonacci Sequence Modulo m

Great question 👍 Let's carefully explain the **Fibonacci Sequence Modulo m** .

◆ Fibonacci Sequence Recap

The Fibonacci sequence is defined as:

$$F_0 = 0, \quad F_1 = 1$$

$$F_n = F_{n-1} + F_{n-2}, \quad n \geq 2$$

So it goes:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

◆ Fibonacci Modulo m

If we take **each Fibonacci number modulo m** , we get a new sequence:

$$F_n \bmod m$$

✓ Example (mod 5):

- $F_0 = 0 \bmod 5 = 0$
- $F_1 = 1 \bmod 5 = 1$
- $F_2 = 1 \bmod 5 = 1$
- $F_3 = 2 \bmod 5 = 2$
- $F_4 = 3 \bmod 5 = 3$
- $F_5 = 5 \bmod 5 = 0$

- $F_6 = 8 \bmod 5 = 3$
- $F_7 = 13 \bmod 5 = 3$
- $F_8 = 21 \bmod 5 = 1$

Sequence mod 5:

0, 1, 1, 2, 3, 0, 3, 3, 1, ...

◆ Key Properties

1. Periodicity (Pisano Period)

The sequence of Fibonacci numbers modulo m is **periodic**.

This repeating cycle is called the **Pisano period**.

For example:

- Mod 2 → cycle: 0,1,1,0,1,1... (period = 3)
- Mod 3 → cycle: 0,1,1,2,0,2,2,1... (period = 8)
- Mod 5 → cycle length = 20

So instead of calculating $F_n \bmod m$ directly, we can reduce n modulo the Pisano period.

2. Efficient Computation

There are two fast ways to compute $F_n \bmod m$:

- **Matrix Method**

Use matrix exponentiation (as explained earlier):

$$\begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^n$$

This gives Fibonacci in $O(\log n)$.

- **Pisano Period**

If we know Pisano period = $\pi(m)$, then:

$$F_n \bmod m = F_{n \bmod \pi(m)} \bmod m$$

So we only compute smaller Fibonacci numbers.

◆ Example Problem

Find $F_{10} \bmod 7$.

1. Fibonacci numbers: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55 ...
2. $F_{10} = 55$
3. $55 \bmod 7 = 6$

✓ Answer: $F_{10} \bmod 7 = 6$

◆ Summary

- Fibonacci modulo m is just taking remainders after each Fibonacci number.
 - The sequence always **repeats** (Pisano period).
 - To compute efficiently for large n , use:
 1. **Matrix exponentiation** ($O(\log n)$)
 2. **Pisano period** (reduce n before computation).
-

👉 Do you want me to also show you **C++ code that directly computes $F_n \bmod m$ efficiently** (using matrix exponentiation)?